



Non-parametric Realtime Rendering of Subspace Objects in Arbitrary Geometric Algebras

Steven De Keninck^(✉)

Matrix Factory, Hingene, Bornem, Belgium
enkimute@gmail.com

Abstract. This paper introduces a novel visualization method for elements of arbitrary Geometric Algebras. The algorithm removes the need for a parametric representation, requires no precomputation, and produces high quality images in realtime. It visualizes the outer product null space (OPNS) of 2-dimensional manifolds directly and uses an isosurface approach to display 1- and 0-dimensional manifolds. A multi-platform browser based implementation is publicly available.

Keywords: Geometric algebra · Implicit visualization · OPNS

1 Introduction

Since its inception, Clifford's Geometric Algebra has become the host of a wide range of algebras, and with it an equally wide range of geometries. They go from the projective $\mathbb{R}_{3,0,1}^*$, (aka PGA [1]), the smallest model with versors covering $SE(3)$, over the popular conformal $\mathbb{R}_{4,1}$ (aka CGA [2,3]), all the way up to $\mathbb{R}_{9,6}$ (aka QCGA [9]), where 9 points in general configuration can be joined into the quadric surface passing through them.

These GA's naturally embed points, lines, planes, spheres, quadrics and much more. Such a wealth of objects comes with its own challenges, one of which is visualization. For a subset of these algebras (notably PGA [1] and CGA [3]), parametric representations of (non degenerate) algebraic elements are well known, and have been the primary method for visualization. (with GAViewer [4] and CLUCalc [5] still being the go-to tools). For higher dimensional algebras the visualization methods used are limited to either raytracing or voxel-based OPNS rendering (see for example [9], 2018). These methods are limited in the items they can render, not realtime, require substantial precomputation, produce low quality images, work only for one algebra, or suffer a combination of these drawbacks.

The algorithm presented in this paper overcomes all of these problems. The quality of its images easily compares to raytracing, without the need for algebra-specific ray-intersection or normal/tangent formulas. Additionally, it can visualize isosurfaces around 1- and 0-dimensional manifolds, as well as non-blade

and/or mixed multivectors. In Sect. 2 we present the algorithm and review some of the implementation details. In Sect. 3 we present some examples, concluding in Sect. 4 (Fig. 1).

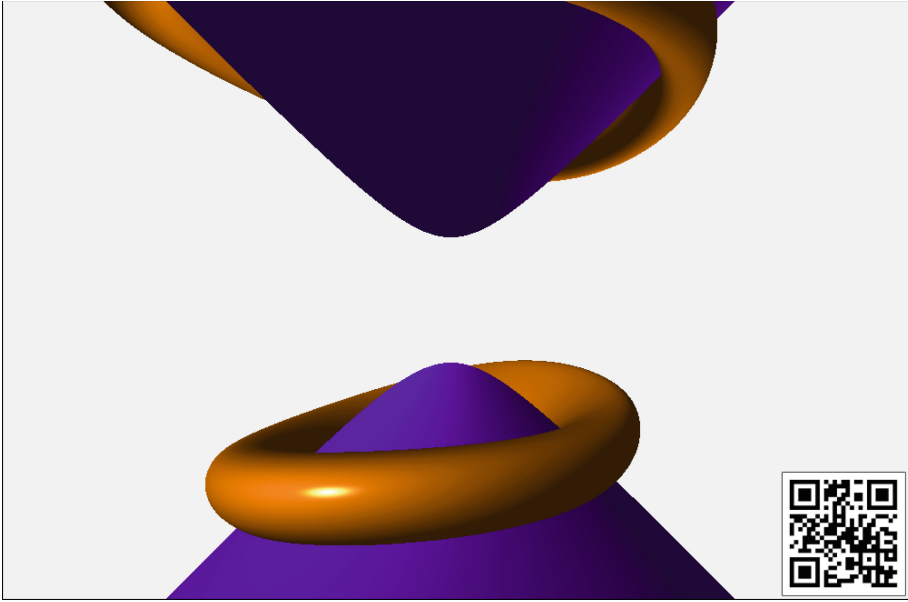


Fig. 1. A quadric (purple) and its intersection (orange) with another quadric (not drawn) in $\mathbb{R}_{9,6}$. (Color figure online)

2 Implicit OPNS Rendering

2.1 OPNS Representation

The graded nature of Clifford's algebras aligns naturally with a geometric interpretation where 0-, 1- and 2-dimensional manifolds embedded in three dimensional Euclidean space are constructed using the outer product. (or depending on the algebra, the regressive ($\vee$) product.)

For these algebras (including PGA [1], CGA [3], MA [10], QCGA [9], ..), a point p lies on the geometric object defined by the multivector $\mathbf{M}$ iff:

$$p \wedge \mathbf{M} = 0 \quad (1)$$

This is the implicit definition of the geometry of $\mathbf{M}$. The collection of all points p for which (1) holds is called the Outer Product Null Space (OPNS). Our algorithm directly visualizes the outer product null space using a ray marching

technique to find the roots of the squared norm of the outer product (where the overline denotes the Clifford conjugate):

$$(p \wedge \mathbf{M})\overline{(p \wedge \mathbf{M})} = 0 \quad (2)$$

The result of (2) is a scalar value that is used to modify the ray-marching step size. For 2-dimensional manifolds embedded in Euclidean 3-space, the result of (2) will be signed, and the ray-marching method can easily be used to detect the sign change as each ray intersects the surface. For 1- and 0-dimensional items, curves and (collections of) points, (2) will not always admit a sign-change, and a threshold value (γ) is used to visualize the isosurface belonging to the shape of these subspaces, by finding the locations p that solve:

$$f(p, \mathbf{M}) = ((p \wedge \mathbf{M})\overline{(p \wedge \mathbf{M})}) - \gamma \quad (3)$$

The metric properties of (3) can vary greatly depending on the algebra, its point parametrization, and the grade of the element being rendered. In all cases however (3) is a continuous smooth function along a display ray and its roots can be found using numerical methods. We propose using (3) as a signed distance bound, and introduce a linear scale α and an exponential scale β to guarantee:

$$\left| (\alpha(p \wedge \mathbf{M})\overline{(p \wedge \mathbf{M})})^\beta - \gamma \right| \leq \min_{q \in \mathbf{M}} \|q - p\| \quad (4)$$

i.e. that the absolute value of the signed distance bound $f(p, \mathbf{M})$ between the point p and the multivector $\mathbf{M}$ is strictly smaller than the distance from p to any of the points that make up $\mathbf{M}$. In practice, the algorithm is robust even for sub-optimal choices for α and β , only affecting rendering performance.

2.2 Sphere Tracing

Originally introduced by Hart [7] in 1994, sphere tracing is now commonly known as signed distance field rendering. Over the past decade, with increased GPU flexibility, the technique has gained popularity, (e.g. [8]), despite its limited scope (it is not trivially suited to render polygonal meshes). It is however well-suited for our purposes. At heart it is a simple modification of the ray marching algorithm, where the step size is adjusted in each iteration based on the value of the signed distance bound. Starting from the camera position, specified by the Euclidean vector $\mathbf{p}_c$, we aim to find the smallest positive distance d along a ray with unit direction vector $\mathbf{r}$ for which the signed distance bound is zero:

$$f(\mathbf{p}_c + d\mathbf{r}, \mathbf{M}) = 0 \quad (5)$$

Starting from $d = 0$, the algorithm updates d each step with the result from the distance bound until it changes sign or is sufficiently close to zero.

$$d_{n+1} = d_n + |f(\mathbf{p}_c + d_n\mathbf{r}, \mathbf{M})| \quad (6)$$

Given our limited space, we refer the interested reader to [7] for more details and instead focus on the challenges specific to the rendering of GA elements. Notably, we will need to convert Euclidean points along the camera ray to their GA equivalent, and calculate the result of (4) without using the storage it would naively require.

2.3 Point Parametrization

With the exception of PGA, point parametrizations in Geometric Algebras are typically non-linear, and we start by adjusting (5) to reflect this change.

$$d_{n+1} = d_n + f(\text{up}(\mathbf{p}_c + d_n \mathbf{r}), \mathbf{M}) \tag{7}$$

where “up” is a multivector valued function of a vector. It is used to convert the point at location $\mathbf{p}_c + d_n \mathbf{r}$ to its GA representation (typically a 1-vector). To facilitate automatic code generation, point parametrizations have to be specified using only the unit basis vectors. (no n_o or n_∞). Table 1 lists those parametrizations for a selection of modern GA’s.

Table 1. Point parametrizations for a selection of GA’s.

Algebra		Parametrization
PGA [1]	$\mathbb{R}_{3,0,1}^*$	$[1, x, y, z]$
CGA [3]	$\mathbb{R}_{4,1}$	$[x, y, z, \frac{1}{2}(x^2 + y^2 + z^2) - \frac{1}{2}, \frac{1}{2}(x^2 + y^2 + z^2) + \frac{1}{2}]$
MA [10]	$\mathbb{R}_{4,4}$	$[\frac{1}{2}, \frac{1}{2}x, \frac{1}{2}y, \frac{1}{2}z, \frac{1}{2}, \frac{1}{2}x, \frac{1}{2}y, \frac{1}{2}z]$
CCGA [11]	$\mathbb{R}_{6,3}$	$[x, y, z, \frac{1}{2}x^2 - \frac{1}{2}, \frac{1}{2}y^2 - \frac{1}{2}, \frac{1}{2}z^2 - \frac{1}{2}, \frac{1}{2}x^2 + \frac{1}{2}, \frac{1}{2}y^2 + \frac{1}{2}, \frac{1}{2}z^2 + \frac{1}{2}]$
QCGA [9]	$\mathbb{R}_{9,6}$	$[x, y, z, \frac{1}{2}x^2 - \frac{1}{2}, \frac{1}{2}y^2 - \frac{1}{2}, \frac{1}{2}z^2 - \frac{1}{2}, \frac{1}{2}xy, \frac{1}{2}xz, \frac{1}{2}yz, \frac{1}{2}x^2 + \frac{1}{2}, \frac{1}{2}y^2 + \frac{1}{2}, \frac{1}{2}z^2 + \frac{1}{2}, xy, xz, yz]$

2.4 Code Generation

A straightforward implementation of the formula in (4) requires storing the result of the outer product $p \wedge \mathbf{M}$. While a graded, sparse format provides substantial savings, in an algebra like QCGA this would still require several hundred components to be stored.

With one ray per pixel and hundreds of these evaluations per ray, a realtime implementation seems impossible on today’s GPU hardware. We note however that the result of the outer product in (4) is only used to calculate the norm, an operation that can be replaced with squaring each coefficient and multiplying it with its weight (the corresponding element on the diagonal of the Cayley table).

If we calculate the norm in this way, we can also reorder the operations for the outer product so that we reduce the storage requirements to just two floating

point variables. One is used to calculate the next component of the outer product while the other accumulates the squared norm. (see listing 1 for an example).

To generate the required GLSL (openGL Shading Language) code, we first added a graded, sparse, numerical GA generator to the `ganja.js` [6] library. This generator was then extended with a symbolic generator capable of dynamically creating the shader code. The source code for these generators is publicly available and we include a listing (listing 1) of the resulting shader that calculates the signed distance bound between a Euclidean point defined by (x, y, z) and a circle in $\mathbb{R}_{4,1}$ CGA. Using the generated distance bound function, implementation of the ray marching algorithm is trivial, and enables us to find the intersection between a ray and an arbitrary multivector.

```
float bound (in float z, in float y, in float x, in float[10] b, in float A,
in float B) {
    float sum; float res;
    res = x*b[6] - y*b[3] + z*b[1] - (.5*(x*x+y*y+z*z)-.5)*b[0];
    sum = res*res;
    res = x*b[7] - y*b[4] + z*b[2] - (.5*(x*x+y*y+z*z)+.5)*b[0];
    sum -= res*res;
    res = x*b[8] - y*b[5] + (.5*(x*x+y*y+z*z)-.5)*b[2] - (.5*(x*x+y*y+z*z)+.5)*b[1];
    sum -= res*res;
    res = x*b[9] - z*b[5] + (.5*(x*x+y*y+z*z)-.5)*b[4] - (.5*(x*x+y*y+z*z)+.5)*b[3];
    sum -= res*res;
    res = y*b[9] - z*b[8] + (.5*(x*x+y*y+z*z)-.5)*b[7] - (.5*(x*x+y*y+z*z)+.5)*b[6];
    sum -= res*res;
    return pow(A*sum,B);
}
```

listing 1. Reordering the outer product components and interleaving the norm calculations allows for a storage-free evaluation (here between CGA circle (given by b) and Euclidean point at location $[x, y, z]$).

2.5 Normals

Finding the point of intersection is however not sufficient for lighting and shading, for which we need to find or approximate the tangent space to the multivector at the point of intersection. Recognizing that our distance bound function is a smooth function (around the intersection point), we exploit the local isomorphy to $\mathbb{R}^3$, and calculate the local normal using the method known as central differences. (see listing 2, the h value is chosen to produce a sub-pixel offset).

```
vec3 n = normalize(vec3(
    bound(d2[0]+h,d2[1],d2[2],b,alpha,beta)-bound(d2[0]-h,d2[1],d2[2],b,alpha,beta),
    bound(d2[0],d2[1]+h,d2[2],b,alpha,beta)-bound(d2[0],d2[1]-h,d2[2],b,alpha,beta),
    bound(d2[0],d2[1],d2[2]+h,b,alpha,beta)-bound(d2[0],d2[1],d2[2]-h,b,alpha,beta)
));
```

listing 2. Using central differences to calculate the normal.

3 Some Examples

3.1 High Dimensional

Using only the parametrizations from Table 1, we produce a number of realtime images in $\mathbb{R}_{4,4}$, $\mathbb{R}_{6,3}$ and $\mathbb{R}_{9,6}$. (4k/60+ fps on nVidia 1080, HD/30+ fps on mobile Adreno 630, for all examples) (Fig. 2).

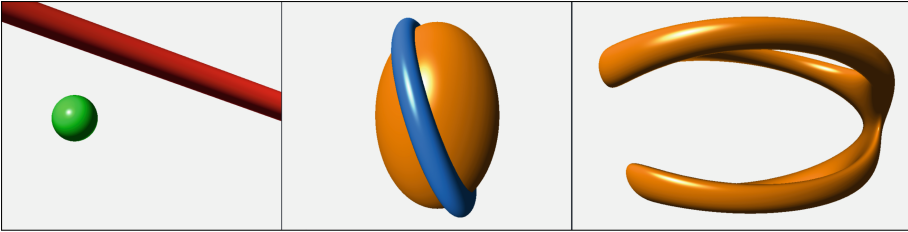


Fig. 2. A line and point in $\mathbb{R}_{4,4}$ (left), an ellipsoid and its intersection with a plane in $\mathbb{R}_{6,3}$ (middle), and a grade-14 element in $\mathbb{R}_{9,6}$ (right).

3.2 Non-Blade Elements

The Algorithm is also capable of rendering the OPNS of non-blade elements. Figure 3 reveals the shape of the subspace of the interpolation between two circles in $\mathbb{R}_{4,1}$ CGA. The author is unaware of any other visualizations of the approximate OPNS of these objects, and we feel this may add the geometric interpretation of these non-object entities. (see e.g. Hadfield and Lasenby [12]).

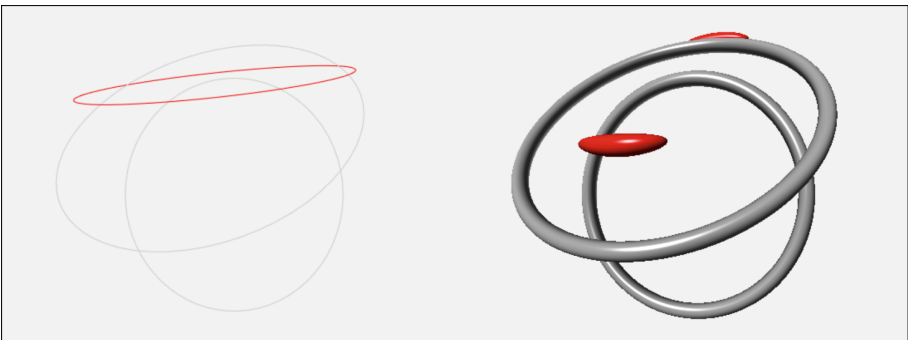


Fig. 3. The interpolation between two circles (gray) produces a non-blade object (red). (left) classic visualization techniques fail to represent the actual shape of the approximate OPNS revealed by our implementation (right) (Color figure online)

4 Conclusion

In this paper we have described a technique that can be used to adapt the Sphere Tracing algorithm to render isosurfaces of elements in arbitrary Geometric Algebras. Avoiding parametrizations or algebra specific ray intersection formulas, the algorithm is readily applicable to even the most challenging algebras, and capable of producing high quality images in real time. It represents a significant increase in flexibility and applicability compared to the visualization methods in use today.

Acknowledgments. The author would like to thank Vincent Nozick for posing the challenge that ultimately led to the implementation presented here, Stephane Breuils for such an exquisite stress test with his QCGA and Leo Dorst for his extensive review of the manuscript. Further thanks go to Charles Gunn and Hugo Hadfield for proofreading.

References

1. Gunn, C.: Geometric algebras for Euclidean geometry. *Adv. Appl. Clifford Algebra* **27**(1), 185–208 (2017). <https://arxiv.org/abs/1411.650>
2. Hestenes, D.: *New Foundations for Classical Mechanics*, 2nd edn. D. Reidel, Dordrecht/Boston (1999)
3. Dorst, L., Fontijne, D., Mann, S.: *Geometric Algebra for Computer Science: An Object-Oriented Approach To Geometry*. Morgan Kaufmann, Burlington (2007)
4. Dorst, L., Fontijne, D., Mann, S.: GAViewer. <http://www.geometricalgebra.net>
5. Perwass, C.: CLUCalc/CLUViz. <http://www.clucalc.info>
6. De Keninck, S.: Ganja.js. *Geometric Algebra - Not Just Algebra* (2017). <https://github.com/enkimute/ganja.js>
7. Hart, J.C.: sphere tracing: a geometric method for the antialiased ray tracing of implicit surfaces (1994). <https://doi.org/10.1007/s003710050084>
8. Inigo Quilez - iq/rgba: Rendering worlds with two triangles. NVSCENE (2008). <http://www.iquilezles.org/www/material/nvscene2008/rwwtt.pdf>
9. Breuils, S., Nozick, V., Sugimoto, A., Hitzer, E.: Quadric conformal geometric algebra of $\mathbb{R}_{4,4}$. *Adv. Appl. Clifford Algebras* **28**, 35 (2018). <https://doi.org/10.1007/s00006-018-0851-1>
10. Juan, D., Goldman, R., Mann, S.: Modeling 3D Geometry in the clifford algebra $\mathbb{R}_{4,4}$. *Adv. Appl. Clifford Algebras* **27**(4), 3039–3062 (2017)
11. Zamora-Esquivel, J.: $G_{6,3}$ geometric algebra; description and implementation. *Adv. Appl. Clifford Algebras* **24**(2), 493–514 (2014)
12. Hadfield, H., Lasenby, J.: Direct linear interpolation of geometric objects in conformal geometric algebra. To be published